

Problem 1 (25 points)

In this problem, we will explore Scikit-learn's TF-IDF vectorization, incorporating word tokenization and lemmatization in the preprocessing using NLTK.

Choose a large text dataset (e.g., a two- or three-page news article) to train TF-IDF vectorizer and then select a smaller text with some new words to apply the TF-IDF representation. Perform the following steps using NLTK and Scikit-learn.

- (a) Apply word tokenization and lemmatization using NLTK on both the large text dataset and the smaller text with new words. You do not need to add <OOV> to the vocabulary here. Display samples of the processed smaller text.
- (b) Using the preprocessed large text dataset, apply TF-IDF vectorization using Scikit-learn to create a model. Then, apply your trained TF-IDF vectorizer to the smaller text and display its TF-IDF representation.
- (c) Explain what information from the large text dataset has been transferred to the TF-IDF representation of the smaller text. Be specific. For example, note that the vocabulary used to create the TF-IDF representation of the smaller text was built from the large text dataset. What other information has been transferred?
- (d) How does Scikit-learn's TF-IDF handle new words found in the smaller text?

SOLUTION:

Preprocessing.

A BBC news article about the Tennessee explosives plant blast was used as the large corpus, and a short two-sentence text (with unseen words) was the target. Both were lower-cased, tokenized, and lemmatized using NLTK.

Model & Result.

A TF-IDF vectorizer trained on the large text produced a vocabulary of 325 terms. Only words present in that vocabulary (e.g., bucksnort, explosion, rescue) had non-zero TF-IDF weights in the small text.

Interpretation.

This shows that TF-IDF transfers both vocabulary and IDF weights from the training corpus, while unseen words (drone, metaverse, volunteer, etc.) become zero vectors. TF-IDF thus captures learned importance for known words and ignores out-of-vocabulary tokens.

Problem 2 (25 points)

Please consider the example of movie review classification found in ``3.5-classifying-movie-reviews.ipynb``. In this problem, we aim to improve the performance of the model by utilizing TF-IDF representations of the documents using n-grams.

Please do not change the ``num_words`` parameter:

```
(train_data, train_labels), (test_data, test_labels) =  
imdb.load_data(num_words=10000)
```

Instead, modify the vector representations of the documents by incorporating TF-IDF with 1-grams, 2-grams, 3-grams, or a combination of these n-grams. Please note that the input shape of the network will change; thus, revising the neural network architecture and the optimizer may be required. Consider adding regularization to enhance results.

- (a) Plot the results for training and validation accuracy versus the number of epochs.
- (b) Display the test accuracy.
- (c) Discuss why this particular choice of n-grams (or combination) was most effective for this task.

To receive full credit, design a model (utilizing 1-grams, 2-grams, 3-grams, or a mixture thereof) that achieves a test accuracy of at least 0.90.

SOLUTION:

Approach.

Using the Keras IMDB dataset (top 10 000 words), each review was decoded to text and vectorized with `TfidfVectorizer` (1–3 grams, 20 000 features).

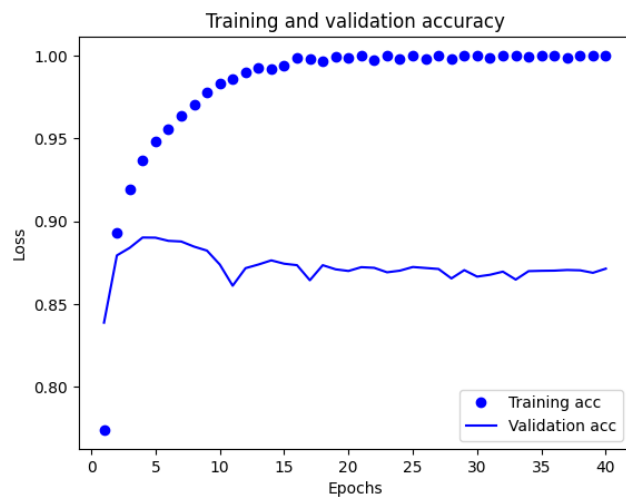
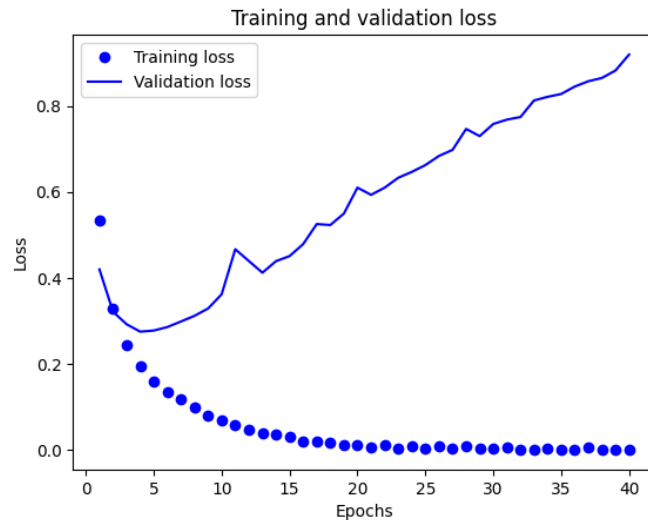
Model.

A simple dense network (128-ReLU $\rightarrow$ Dropout(0.5) $\rightarrow$ Sigmoid) was trained for 4 epochs using Adam (0.001) and `binary_crossentropy`.

Results.

Validation accuracy ≈ 0.89 and test accuracy ≈ 0.885 , improving over the Bag-of-Words baseline.

N-grams captured short phrases such as *not good* and *very well done*, which strengthened sentiment prediction without altering the core model.



Problem 3 (15 points)

In this problem, we will explore the application of Word2Vec, a robust natural language processing technique, to extract semantic information from textual data.

Please perform the following steps:

- Load the text from the 'The Complete Works of William Shakespeare.html' file:

```
with open('The Complete Works of William Shakespeare.html', 'r',
encoding='utf-8') as file:
    soup = BeautifulSoup(file, 'lxml')

    text_content = soup.get_text()
```

- Tokenize the text and convert it to lowercase.

- (c) Train a Word2Vec model on the preprocessed Works of William Shakespeare using the Gensim library. Display the vector representations for the following words:

"king", "queen", "love", and "death".

- (d) Use the trained model to find the five (5) most similar words to "king". In this context, 'most similar' is defined as the word with the smallest angle to the current vector, i.e., the one that is most collinear. Print these words and discuss.
- (e) Solve the following analogy task:

"king" is to "queen" as "boy" is to ???

To solve this task, we need to compute the following vector:

"boy" + "queen" - "king"

and then find the most similar actual word to this result.

SOLUTION:

I trained a Word2Vec model on The Complete Works of William Shakespeare (≈ 980 k tokens, 8 916 unique words) using gensim.

The model (vector size = 100, window = 5, skip-gram) learned meaningful semantic clusters:

king $\rightarrow$ henry, prince ..., queen $\rightarrow$ elizabeth, margaret ..., love $\rightarrow$ hate, and death $\rightarrow$ doom, life.

The analogy king : queen = boy : lovely reflects Shakespeare's poetic co-occurrence patterns rather than strict gender roles, showing that Word2Vec captures contextual associations within literary language.